

2º curso / 2º
cuatrimestre.

Grado en
Ingeniería
Informática

Arquitectura de Computadores (AC)

Cuaderno de prácticas.

Bloque Práctico 4. Optimización de código

Estudiante (nombre y apellidos): David Rodríguez Aparicio

Marca del procesador (se encuentra en /proc/cpuinfo y se lista con lscpu): 12th Gen Intel(R) Core(TM) i7-12650H

Sistema operativo: Windows 11 -> Ubuntu 24.04.4 LTS (WSL desde VSCODE)

Versión de gcc utilizada: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

1. Modificar el código secuencial dado para la multiplicación de matrices de forma que el producto se calcule más rápidamente, reduciendo el tiempo de ejecución (centrarse sólo en implementar la función “fastProduct”). Justificar los tiempos obtenidos (usando siempre “-O2”) a partir de la modificación realizada. Incorporar la mejor versión de “fastProduct” desarrollada al cuaderno.

MODIFICACIONES REALIZADAS (al menos dos modificaciones):

Modificación A) –explicación–: Se ha cambiado el orden de los bucles de i-j-k a i-k-j para mejorar la localidad espacial de memoria. La matriz m2 pasa a recorrerse por filas, que es el orden en que se almacena en C, reduciendo fallos de caché y mejorando el rendimiento.

Modificación B) –explicación–:

Se aplica desenrollado del bucle interno (loop unrolling) procesando cuatro elementos por iteración. Esto reduce el número de saltos y permite al procesador aprovechar mejor el paralelismo a nivel de instrucción (ILP).

CÓDIGO DEL CÁLCULO MEJORADO:

A) Captura del código para la primera modificación (función “fastProduct”)

```
void fastProduct(int N, double** m1, double** m2, double** m3) {
    for(int i = 0; i < N; i++) {
        for(int k = 0; k < N; k++) {
            double temp = m1[i][k];

            for(int j = 0; j < N; j++) {
                m3[i][j] += temp * m2[k][j];
            }
        }
    }
}
```

Capturas de pantalla (que muestren la compilación y que el resultado es correcto):

```
Matriz 1 (m1):  
[0][0] = 1.00, (...), [499][499] = 1.00  
Matriz 2 (m2):  
[0][0] = 2.00, (...), [499][499] = 2.00  
  
Version inicial:  
Tiempo: 0.156569162      Tamaño: 500  
Matriz Resultado (m3):  
[0][0] = 1000.00, (...), [499][499] = 1000.00  
-----
```

```
Version mejorada:  
Tiempo: 0.076475407      Tamaño: 500  
Matriz Resultado (m3):  
[0][0] = 1000.00, (...), [499][499] = 1000.00  
-----
```

```
Matriz 1 (m1):  
[0][0] = 1.00, (...), [999][999] = 1.00  
Matriz 2 (m2):  
[0][0] = 2.00, (...), [999][999] = 2.00  
  
Version inicial:  
Tiempo: 1.494402582      Tamaño: 1000  
Matriz Resultado (m3):  
[0][0] = 2000.00, (...), [999][999] = 2000.00  
-----
```

```
Version mejorada:  
Tiempo: 0.537959843      Tamaño: 1000  
Matriz Resultado (m3):  
[0][0] = 2000.00, (...), [999][999] = 2000.00  
-----
```

```

Matriz 1 (m1):
[0][0] = 1.00, (...), [1499][1499] = 1.00
Matriz 2 (m2):
[0][0] = 2.00, (...), [1499][1499] = 2.00

Version inicial:
Tiempo: 6.345504580      Tamaño: 1500
Matriz Resultado (m3):
[0][0] = 3000.00, (...), [1499][1499] = 3000.00
-----

Version mejorada:
Tiempo: 2.999309542      Tamaño: 1500
Matriz Resultado (m3):
[0][0] = 3000.00, (...), [1499][1499] = 3000.00
-----

```

B) Captura del código para la segunda modificación (función “*fastProduct*”)

```

void fastProduct(int N, double** m1, double** m2, double** m3){
    for(int i = 0; i < N; i++){

        for(int k = 0; k < N; k++){

            double temp = m1[i][k];

            int j;

            for(j = 0; j <= N - 4; j += 4){
                m3[i][j]      += temp * m2[k][j];
                m3[i][j + 1] += temp * m2[k][j + 1];
                m3[i][j + 2] += temp * m2[k][j + 2];
                m3[i][j + 3] += temp * m2[k][j + 3];
            }

            for(; j < N; j++){
                m3[i][j] += temp * m2[k][j];
            }

        }

    }
}

```

```

david@LP-DAVID15:/mnt/c/Users/david/Desktop/Ingenieria-Informatica-2/AC/Practica 4 (+ S4)$ ./pmm 1500
Matriz 1 (m1):
[0][0] = 1.00, (...), [1499][1499] = 1.00
Matriz 2 (m2):
[0][0] = 2.00, (...), [1499][1499] = 2.00

Version inicial:
Tiempo: 5.893559515      Tamaño: 1500
Matriz Resultado (m3):
[0][0] = 3000.00, (...), [1499][1499] = 3000.00
-----

Version mejorada:
Tiempo: 1.858939889      Tamaño: 1500
Matriz Resultado (m3):
[0][0] = 3000.00, (...), [1499][1499] = 3000.00
-----

```

TIEMPOS:

Modificación	Breve descripción de las modificaciones	-O2
Sin modificar	<i>Multiplicación clásica de matrices con orden de bucles i-j-k.</i>	5,894 s
Modificación A)	Cambio del orden de los bucles a i-k-j para mejorar la localidad de memoria y el aprovechamiento de la caché.	2,999 s
Modificación B)	Modificación A + desenrollado del bucle interno (loop unrolling ×4) para reducir saltos y aumentar el paralelismo.	1,859 s
...		

COMENTARIOS SOBRE LOS RESULTADOS Y JUSTIFICACIÓN DE LAS MEJORAS EN TIEMPO:

Se observa que la versión modificada mejora bastante el tiempo respecto a la original.

La Modificación A mejora el acceso a memoria cambiando el orden de los bucles.

La Modificación B añade desenrollado del bucle interno, reduciendo saltos y aprovechando mejor el procesador.

La mejor versión es la B, ya que baja el tiempo de 5,894 s a 1,859 s.

2. Usar en este ejercicio el programa secuencial disponible en la plataforma que utiliza como base el código de la Figura 1. Modificar en el programa el código mostrado en la Figura 1 para reducir el tiempo de ejecución. Justificar los tiempos obtenidos (usando siempre “-O2”) a partir de la modificación realizada. En las ejecuciones de evaluación usar valores de N y M mayores que 1000. Incorporar los códigos modificados en el cuaderno.

Figura 1 . Código C++ que suma dos vectores. M y N deben ser parámetros de entrada al programa, usar valores mayores que 1000 en la evaluación.

```
struct nombre {
    int a;
    int b;
} s[N];

main()
{
    ...
    for (ii=0; ii<M;ii++) {
        X1=0; X2=0;
        for(i=0; i<N;i++) X1+=2*s[i].a+ii;
        for(i=0; i<N;i++) X2+=3*s[i].b-ii;

        if (X1<X2) R[ii]=X1 else R[ii]=X2;
    }
    ...
}
```

MODIFICACIONES REALIZADAS (al menos dos modificaciones):

Modificación A) –explicación–: Se han fusionado los dos bucles que recorrían el vector en un único recorrido, reduciendo el número de accesos a memoria.

Modificación B) –explicación–: Además de fusionar los bucles, se han sacado fuera del bucle interno las operaciones constantes que dependen de *ii*, reduciendo operaciones repetidas.

CÓDIGOS FUENTE MODIFICACIONES

A) Captura figura1-modificado_A.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct sestruct{
```

```
    int a;
    int b;
} SetStruct;

int main(int argc, char** argv){
    struct timespec cgt1,cgt2;
    double ncgt;
    int ii, i, X1, X2;

    if (argc<3){
        printf("Faltan argumentos de entrada (N, M)");
        exit(-1);
    }

    int N = atoi(argv[1]);
    int M = atoi(argv[2]);

    SetStruct s[N];
    int R[M];

    if(N<9 && M<9){
        for(i=0;i<N;i++){
            s[i].a=-i;
            s[i].b=i;
        }
    }else{
        srand(time(0));
        for(i=0;i<N;i++){
            s[i].a=-rand() %200;
            s[i].b=rand() %100;
        }
    }

    clock_gettime(CLOCK_MONOTONIC, &cgt1);

    for (ii=0; ii<M; ii++){

        X1=0;
        X2=0;

        for(i=0; i<N; i++){
```

```

        X1 += 2*s[i].a + ii;
        X2 += 3*s[i].b - ii;
    }

    if (X1<X2)
        R[ii]=X1;
    else
        R[ii]=X2;
}

clock_gettime(CLOCK_MONOTONIC, &cgt2);

ncgt = (double) (cgt2.tv_sec-cgt1.tv_sec)
      + (double) ((cgt2.tv_nsec-cgt1.tv_nsec)/(1.e+9));

if(N<10 && M<10) {
    for(i=0;i<N;i++) {
        printf("s[%d].a=%d \t s[%d].b=%d \n", i, s[i].a, i, s[i].b);
    }
    for(i=0;i<M;i++) {
        printf("R[%d]=%d \t", i, R[i]);
    }
    printf("\n");
} else {
    printf("Tiempo: %11.9f\n", ncgt);
    printf("Elemento 0 y %d de R: %d, %d\n", M-1, R[0], R[M-1]);
}

return 0;
}

```

Capturas de pantalla (que muestren la compilación y que el resultado es correcto):

B) #include <stdio.h>

#include <stdlib.h>

#include <time.h>

typedef struct sestruct{

int a;

```
    int b;
} SetStruct;

int main(int argc, char** argv){

    struct timespec cgt1, cgt2;

    double ncgt;

    int ii, i, X1, X2;

    if (argc<3){

        printf("Faltan argumentos de entrada (N, M)");

        exit(-1);

    }

    int N = atoi(argv[1]);
    int M = atoi(argv[2]);

    SetStruct s[N];
    int R[M];

    if(N<9 && M<9){

        for(i=0; i<N; i++){

            s[i].a=-i;

            s[i].b=i;

        }

    }else{

        srand(time(0));

        for(i=0; i<N; i++){

            s[i].a=-rand() %200;

            s[i].b=rand() %100;

        }

    }

}
```



```

clock_gettime(CLOCK_MONOTONIC, &cgt1);

for (ii=0; ii<M; ii++) {

    X1=0;
    X2=0;

    for(i=0; i<N; i++) {
        X1 += 2*s[i].a;
        X2 += 3*s[i].b;
    }

    X1 += N*ii;
    X2 -= N*ii;

    R[ii] = (X1 < X2) ? X1 : X2;
}

clock_gettime(CLOCK_MONOTONIC, &cgt2);

ncgt = (double) (cgt2.tv_sec-cgt1.tv_sec)
        + (double) ((cgt2.tv_nsec-cgt1.tv_nsec)/(1.e+9));

if(N<10 && M<10) {
    for(i=0;i<N;i++) {
        printf("s[%d].a=%d \t s[%d].b=%d \n", i, s[i].a, i, s[i].b);
    }
    for(i=0;i<M;i++) {
        printf("R[%d]=%d \t", i, R[i]);
    }
    printf("\n");
}

```

```

    }else{

        printf("Tiempo: %11.9f\n", ncgt);

        printf("Elemento 0 y %d de R: %d, %d\n", M-1, R[0], R[M-1]);

    }

    return 0;

}

```

```

david@LP-DAVID15:/mnt/c/Users/david/Desktop/Ingenieria-Informatica-2/AC/Practica 4 (+ S4)$ ./fig1 5000 5000
./fig1A 5000 5000
./fig1B 5000 5000
Tiempo: 0.024011749
Elemento 0 y 4999 de R: -998938, -24247379
Tiempo: 0.021673510
Elemento 0 y 4999 de R: -998938, -24247379
Tiempo: 0.022828793
Elemento 0 y 4999 de R: -998938, -24247379

```

TIEMPOS:

Modificación	Breve descripción de las modificaciones	-O2
Sin modificar	<i>Dos recorridos independientes del vector s.</i>	<i>0.024012 s</i>
Modificación A)	Fusión de los dos bucles en un único recorrido del vector.	0.021674 s
Modificación B)	Modificación A + eliminación de operaciones repetidas dentro del bucle.	0.022829 s
...		

COMENTARIOS SOBRE LOS RESULTADOS Y JUSTIFICACIÓN DE LAS MEJORAS EN TIEMPO:

La Modificación A mejora ligeramente el tiempo de ejecución al recorrer el vector una sola vez en lugar de dos. Esto reduce los accesos a memoria y el trabajo realizado en cada iteración. La Modificación B también funciona correctamente, aunque en este caso no mejora los resultados obtenidos con la Modificación A. La mejor versión es la A, ya que obtiene el menor tiempo de ejecución.